多模态大模型 八股知识问答（华为 AI 模型工程师岗）

> **说明**：本文档为对外分享版，前雇主项目的量化指标已做量级化处理，方法论与话术保持原样。
> 与《面试速成指南》第 5 章的 12 问速记版配套：**那份是口头速记，这份是完整答案库**。 共 **74 问**，按面试出现顺序编排。标注说明：★★★ = 几乎必问，★★ = 大概率问，★ = 加分题。 答法原则：**先给一句结论，再给一层原因，最后给一个你亲手做过的证据**。三段式，别背长段落。

S1 视觉编码器与 token（8 问）

Q1 ★★★ ViT 是怎么工作的？一张图变成多少 token 怎么算？

A：把图像切成固定大小的 patch（如 16×16），每个 patch 展平后线性投影成一个 token，加上位置编码，送入标准 Transformer 编码器。token 数 = (H/p) × (W/p)。

现场可背的例子： - 224×224、patch 16 → 14×14 = **196 个 token** - 384×704（车载常见分辨率）、patch 16 → 24×44 = **1056 个 token** - 384×384、patch 14 → 27×27 = **729 个 token**

追问：为什么 patch 越小越好但也越贵？→ token 数平方增长，attention 是 $O(N^2)$，显存和算力都吃紧；SAM/LLaVA-1.6 用更小 patch 换细节，代价是 token 爆炸，所以要配 token 压缩。

Q2 ★★ CLS token 是干什么的？必须有吗？

A：ViT 沿用 BERT 的做法，在序列最前面加一个可学习的 `[CLS]` token，靠 self-attention 汇聚全局信息，做分类时只取它的输出。**不是必须的**：ViT 论文里也试过对 patch token 做全局平均池化（GAP），效果接近；现代多模态模型大多**不用 CLS**，直接把所有 patch token 交给 LLM（因为下游要的是每个位置的细粒度特征，不是单一向量）。

Q3 ★★ ViT 和 CNN 的本质区别？为什么大模型都用 ViT？

A：CNN 有强归纳偏置（局部性、平移等变、权重共享），小数据下更强；ViT 只有全局注意力、没有这些先验，**必须有大数据或强增广/蒸馏才能超过 CNN**，但一旦数据够，它的表达能力和"和 LLM 同构"的优势就出来了。

用 ViT 的真实原因：① 结构同构，方便和多模态 LLM 拼装；② 全局感受野，适合需要长程关系的任务；③ scaling 行为好（参数/数据同比例涨，loss 平滑下降）。

追问：Swin 怎么补回局部性？→ 窗口内注意力 + 窗口移位（shifted window）做跨窗交互 + 分层下采样，复杂度从 $O(N^2)$ 降到 O(N)。

Q4 ★★ 多模态里位置编码有哪些选择？RoPE 为什么能外推？

A：四类——正弦绝对编码、可学习绝对编码、**RoPE（旋转位置编码）**、ALiBi（按距离线性衰减 bias）。

RoPE 的做法：把 q/k 的相邻两维看成一个二维向量，按位置 m 旋转 m・θ 角度，于是内积只依赖相对位置差 (m-n)。**能外推是因为它编码的是相对距离，不是绝对下标**；但直接外推会随距离增大而失真，所以要配位置内插（PI）、NTK-aware 缩放、YaRN 这些方法扩上下文。

追问：图像 token 用 2D RoPE 怎么搞？→ 行列各一套频率（Qwen2-VL 就是 M-RoPE：时间/高/宽三套）。

Q5 ★★ 高分辨率图像怎么处理？（这是多模态工程的核心问题之一）

A：四条主流路线： 1. **固定放大**（LLaVA-1.5 用 336×336 或 448×448）——简单但细节丢失、宽高比失真。 2. **切块 + 缩略图**（LLaVA-1.6/InternVL 的 dynamic tiling）：把大图切成若干 336 块，每块独立编码，再加一张全局缩略图。token 数变成块数×(块 token)+缩略图 token。 3. **原生动态分辨率**（NaViT、Qwen-VL 的 M-RoPE）：直接按原分辨率切 patch，不缩放。 4. **像素重排/pixel shuffle**（InternVL）：把相邻 2×2 个 patch 在通道维拼起来，token 数直接降 4 倍，用空间换通道。

追问：token 变多有什么代价？→ ① LLM 的 prefill 变慢、KV cache 变大；② 图文 token 比例失衡，注意力被图像挤占。

Q6 ★★★ 视觉 token 太多怎么压缩?

A：五类手段，按"是否可学习"排： 1. **池化/重采样**（平均池化、Perceiver Resampler 用固定数量 query 做 cross-attention）——**Q-Former/Perceiver 就是这一类，把可变长 patch 压成固定 32/64 个 token**。 2. **token merge**（相邻相似 token 合并，如 Token Merging、LLaVA-PruMerge）。 3. **空间重排**（pixel shuffle，2×2 拼通道，token ÷4）。 4. **重要性剪枝**（按注意力分数保留 top-k token）。 5. **只取关键帧/区域**（视频/大图场景）。

你的接口话术："我在 BEV 里做的就是同类问题——7 个视角的 patch token 经深度分布加权投影到 65,536 个 BEV query 上，本质是几何约束下的 token 重组；换成 VLM 就是把 query 数量压到 LLM 预算内。"（**这句是本文档最有价值的一句话，务必背下来**）

Q7 ★ ViT 的预训练方式有哪些?

A：① 监督分类（ImageNet）；② **对比学习**（CLIP 图文对比、DINO 自蒸馏无标签）；③ **掩码重建**（MAE，遮 75% patch 重建像素，训练效率高）；④ 生成式（BEiT 预测离散 visual token）。多模态场景主流是 CLIP/SigLIP 这类图文对比预训练，因为它天然对齐了文本空间。

Q8 ★★ 一张 4K 图（3840×2160）patch 14 直接切，token 多少？为什么不能这么干?

A：274×154 ≈ **42,196 个 token**。对比：LLaMA-2 的上下文是 4096。所以不可能直接喂——必须缩放、切块、压缩三选一（见 Q5/Q6）。这也是"端侧多模态"最现实的约束：**分辨率是 token 的来源，token 是算力和显存的货币**。

S2 图文对齐与经典模型（12 问）

Q9 ★★★ CLIP 是怎么训练的？损失函数是什么?

A：双塔（图像塔 ViT/ResNet + 文本塔 Transformer），各自编码后投影到同一维度并 L2 归一化，得到 B×B 的相似度矩阵。**对角是正样本对，其他 2(B-1) 个都是负样本（in-batch negatives）**，损失是对称的 InfoNCE：

```
L = 1/2 · [ CE(图像→文本) + CE(文本→图像) ]
```

即图像侧做一次 softmax 分类（正确文本是 label），文本侧再做一次。温度 τ 是可学习标量（CLIP 里初始化为 0.07 的倒数形式，即 logit_scale=1/τ）。

追问：为什么必须有 in-batch negatives? → 没有负样本，模型会把所有图文映射到同一点（坍缩）。负样本的多样性直接决定表征质量，所以**大 batch 是 CLIP 的关键**（原始实现 batch 32768）。

Q10 ★★ CLIP 怎么零样本分类?

A：把每个类名套进 prompt 模板（如 "a photo of a {class}"）编码成文本向量，图像向量与所有类文本向量算余弦相似度，取最大。**prompt 工程很关键**（CLIP 论文用了 80 个模板做集成），因为文本侧的训练分布是"句子"而非"单词"。

Q11 ★★ CLIP 的局限有哪些?

A：① 只做**全局粗粒度**对齐，不擅长细粒度属性、计数、空间关系（"左边的红球"）；② 词序不敏感（bag-of-words 倾向，"dog bites man" 和 "man bites dog" 难分）；③ 需要海量数据、大 batch；④ 只能做判别式检索，不能生成；⑤ 对分布外/罕见概念泛化差。

追问：怎么补? → 加 region-level 监督（GLIP/RegionCLIP）、加 captioning loss（BLIP）、交给能生成的多模态 LLM。

Q12 ★★ SigLIP 和 CLIP 的区别?

A：把 softmax 的**多分类交叉熵换成逐对 sigmoid 二分类（pairwise sigmoid loss）**。好处：每个图文对独立判断，不再需要全 batch 归一化，**因此 batch 可以小、也能做分布式异步**，训练更省内存；效果同级甚至更好，是现在很多开源 VLM（PaliGemma、Qwen-VL 系）的视觉塔选择。

Q13 ★ BLIP-2 的 Q-Former 是干什么的？为什么要有它?

A：Q-Former 是一个小 Transformer，内含**固定数量（32/64 个）可学习的 query**，通过 cross-attention 从冻结的图像编码器抽取信息，输出固定长度的视觉 token。

它解决两个问题：① **把可变长 patch token 压成固定长度**，让 LLM 的上下文可控；② **在冻结视觉塔和冻结 LLM 的前提下做桥接**，只训这一个轻量模块，成本极低。

两阶段训练：**阶段一表征学习**（ITC 图文对比 + ITM 匹配 + ITG 生成，用 query 学对齐）；**阶段二生成学习**（把 query 输出接到冻结 LLM，用语言建模损失训）。

追问：Q-Former 和 Perceiver Resampler 有什么区别？→ 同一类东西（固定 query + cross-attention），Flamingo 的 Perceiver 是纯重采样后用 gated cross-attention 注入，BLIP-2 的 Q-Former 还要多任务监督、对齐得更强。

Q14 ★★★ LLaVA 为什么用两层 MLP 就够了？这和 Q-Former 的取舍是什么？

A：LLaVA 的消融显示：**在冻结视觉塔 + 冻结 LLM 的设定下，简单线性/MLP 投影和 Q-Former 效果接近甚至更好**，而且更简单、训练更快。原因是视觉塔已经产出了足够好的语义特征，桥接层只需要做"维度对齐 + 轻量变换"，复杂的压缩反而损失信息。

取舍：MLP 保信息但 token 多（贵）；Q-Former 压 token 但要多阶段训练、实现复杂。**决定因素是你的 token 预算**：预算宽松（图少、上下文长）用 MLP；预算紧（视频、端侧）用 query 压缩。

LLaVA 两阶段：① 冻结 LLM 只训 projector（用图文对做"描述生成"式对齐）；② 解冻 LLM 做指令微调（LLaVA-1.5 约 558K 预训练 + 665K 指令数据）。

Q15 ★★ Flamingo 的结构有什么特别?

A：三件套——**冻结的视觉编码器**（NFNet）+ **Perceiver Resampler**（把任意数量视觉特征压成固定数量）+ **在冻结 LLM 里插入 gated cross-attention 层**（用 tanh 门控初始化 0，保证训练初期不破坏原 LLM 行为，这是很漂亮的设计）。它支持交错图文（多图 + 多轮）输入，是小样本多模态能力的代表。

Q16 ★ 为什么大家都冻结视觉塔?

A：① 视觉塔参数量大（ViT-L 约 300M），冻结省显存和算力；② 预训练视觉特征已经很通用，微调反而容易过拟合到指令数据的分布；③ 冻结后视觉侧可以离线预抽特征（或者用更小的学习率）。**什么时候该解冻**：目标任务和预训练分布差很远（医学影像、红外、深度图）、或者需要极细粒度定位时，常见做法是解冻最后几层 + 小学习率。

Q17 ★ 图文匹配里常见的三种 loss 是什么?

A：**ITC**（image-text contrastive，对比学习，学全局对齐）、**ITM**（image-text matching，二分类判断是否匹配，需要融合后的细粒度特征，常用 hard negative 挖掘）、**ITG**（image-grounded text generation，给定图像生成文本，用 causal mask 防止信息泄漏）。BLIP 就是三者联合，各自的 attention mask 不同——这是它设计上的巧思。

Q18 ★ 图文对数据从哪来？质量怎么保证?

A：来源：网页 alt-text 抓取（量最大、噪声也最大）、人工标注（小但准）、合成（用强模型给图生成 caption）、业务数据（你自己的场景）。质量控制三板斧：① **过滤**（短文本、非自然语言、重复、比例异常图、水印图）；② **重写 caption**（用强 VLM 重新描述，把 alt-text 的噪声换成结构化描述，这就是 LLaVA-1.5 / ShareGPT4V 的做法）；③ **去重**（图像近重复是评测泄漏的最大来源）。

你的接口：这块你极强——数万 个场景、百万级 个关键帧的真值生产线，口径核查、版本反转 bug、双 seed 噪声地板，这些方法论完全可以直接讲成"大规模多模态数据的生产线与质量治理"。

Q19 ★★ 如果图像和文本的 token 在序列里混排，怎么区分它们?

A：靠**特殊 token + 位置/模态标识**：用 `<image>` / `<|image_pad|>` 这类特殊占位 token 标出图像区间；位置编码上，文本用文本位置，视觉 token 用 2D（或 3D）位置，或者干脆共享一个序列位置（多数实现）。更细的做法是加**模态嵌入**（modality embedding）或**视觉 token 的类型标记**，让模型知道这段是"看来的"不是"读来的"。

Q20 ★★★ 多模态里的"对齐"到底是哪几层对齐?

A：三层，面试时按层说很清晰： 1. **表征对齐**（representation）：图像和文本映射到同一语义空间——CLIP 干这个。 2. **粒度对齐**（granularity/region）：把文本里的"猫"对应到图像里的某个区域——Q-Former、grounding 数据、RefCOCO 监督干这个。 3. **行为对齐**（behavior）：让模型按人类偏好/指令回答——SFT + RLHF/DPO 干这个。

你的接口："我做的视觉-激光融合是第一层的跨模态对齐（不同传感器到统一 BEV 空间），配合几何标定做第二层的空间对齐；多模态 LLM 的第三层行为对齐是我要补的。"

S3 多模态大模型架构与训练（12 问）

Q21 ★★★ 一个多模态 LLM 由哪几部分组成？

A：三件套 + 两处细节： 1. **视觉编码器**（ViT/SigLIP）——把图变成 token； 2. **模态桥接层**（projector / Q-Former / resampler）——维度与数量对齐； 3. **语言模型**（LLM）——统一在 token 序列上做自回归。

细节：图像/文本的**特殊占位 token** 与位置编码方案；训练时的**冻结/解冻策略**。数据流是 `image → ViT → 桥接 → 与文本 token 拼接 → LLM → 文本输出`。

Q22 ★★★ VLM 的训练分几个阶段？每阶段干什么？

A：三阶段，每阶段的"目的—数据—冻结策略"都不同：

阶段	目的	典型数据量	冻结策略	损失
① 预训练/对齐	让视觉特征能被 LLM 读懂	数十万~数百万图文对（caption 形式）	冻结 ViT + LLM，**只训 projector**	语言建模（生成 caption）
② 指令微调 SFT	学会按指令回答、多轮、拒答	数十万~百万条指令数据	解冻 LLM（或加 LoRA），ViT 常冻结	只在回答部分算 LM loss
③ 偏好对齐	对齐人类偏好、减少幻觉	数万~数十万偏好对	全参或 LoRA	DPO / PPO / GRPO

关键点（加分）：SFT 时**只对 assistant 回复算 loss，用户提问部分 mask 掉**；阶段 ① 用冻结 LLM 是为了不让随机初始化的 projector 把 LLM 带崩。

Q23 ★★ 指令数据怎么造？

A：四条来源：① 人工标注（质量最高、最贵，如 LLaVA 的 GPT-4 生成 + 人工筛）；② 强模型蒸馏（GPT-4V 生成对话，注意**只用它生成文本、不用它生成图像**）；③ 现有数据集转换（VQA、检测、OCR、grounding 数据改写成对话）；④ 合成负样本/拒答样本（教会模型说"图里没有"）。质量治理：去重、任务配比、难度分层、答案长度控制、**加入纯文本数据防遗忘**。

Q24 ★★ 冻结策略怎么选？

A：三种常见配置： - **只训 projector**（最省，适合对齐阶段、大量数据但算力紧）； - **训 projector + LLM（LoRA 或全参）**（主流，SFT 阶段）； - **全解冻含视觉塔**（数据量大、领域差异大时才做，学习率要小，否则破坏预训练特征）。

判断依据一句话：**数据量小就多冻结，领域差异大就多解冻**。

Q25 ★★★ LoRA 是什么？为什么能省显存？

A：冻结原权重 W，只学低秩增量 ΔW = B · A，其中 A∈R^{r×d}、B∈R^{d×r}，r 远小于 d，前向是 `h = Wx + (alpha/r) · BAx`。

省显存的原因：**只对 A/B 求梯度、只给它们存优化器状态**（Adam 要 2 份动量，这才是显存大头），可训练参数常降到 0.1%~1%。推理时可以把 BA 加回 W，**零额外延迟**。

参数建议：rank 8~64，alpha 常取 2r，作用于 q/k/v/o 和 MLP 的全部线性层（比只加 q/v 效果好）。追问 **QLoRA**：基座量化到 4bit（NF4）+ 双重量化 + paged optimizer，可在单卡训 65B。

Q26 ★★★ 训练大模型显存都花在哪？怎么省？

A：四块（按大小排，以参数量 P、字节数计）：**参数 2P（fp16）+ 梯度 2P + 优化器状态 12P（Adam fp32 的 m/v/主权重）+ 激活**。前三个是"静态的"，激活随 batch×序列长度线性增长。

省的手段：**ZeRO-1 切优化器状态 / ZeRO-2 再切梯度 / ZeRO-3 再切参数**；**梯度检查点**（重算激活，显存换 30% 时间）；**混合精度 + bf16**（比 fp16 稳，不用 loss scaling）；**梯度累积**（小 batch 模拟大 batch）；**序列并行/FlashAttention**（省激活里的注意力矩阵）；**offload**（CPU/NVMe，慢）。

你的接口："我在 8 卡训练集群 上把显存从 约 89GB 压到 约 82GB，方法是用 32×32 混合矩阵替代 3-tap 大核，保住 batch 和分辨率——本质是'算力不变但中间张量变小'的重构思路。"（**这条极其对口，面试官会顺着问下去**）

Q27 ★★ 训练不稳定 / loss 突然爆掉 / 出 nan，怎么排查?

A：按顺序查，别乱猜： 1. **数据侧**：label 越界、空样本、异常值（我遇到的真实案例：CE 的 target 越界，参数名 `cls_num` 与实际类别数不一致，loss 直接崩）； 2. **数值侧**：fp16 溢出 → 换 bf16 或加 loss scaling；attention 里 mask 全 −inf → nan；除零； 3. **学习率**：太大 → 加 warmup、降峰值；梯度裁剪（通常 1.0）； 4. **结构侧**：新加的层初始化不当（用恒等初始化/零门控）；残差尺度； 5. **硬件侧**：CUBLAS workspace 不足、OOM 报错伪装成算子错误（我真遇到过 `CUBLAS_STATUS_ALLOC_FAILED`，根因是超大 batch 的 GEMM workspace，改 depthwise+pool 解决）。

答法价值：这段回答的分数在"你怎么定位"，而不是"你修了什么"——按"数据→数值→超参→结构→硬件"的层次说，显得有方法论。

Q28 ★ 多图/多轮对话在序列里怎么组织?

A：图像 token 按出现顺序插在对话流里，用特殊 token 界定每张图的区间；多轮时**历史轮次保留（可能要压缩）**，图像通常只在第一次出现时插入，后续轮引用"上面那张图"。工程上的三个坑：① KV cache 随轮数增长；② 图像 token 太多导致历史被挤掉，需要"图像 token 压缩/丢弃旧图"策略；③ 位置编码要能区分"第几张图"。

Q29 ★★ 宽高比差异大的图（比如 1920×200 的长条票据）怎么处理?

A：固定 resize 会严重失真（OCR 直接废掉）。方案：**按比例切块**（切成若干接近正方形的小块 + 一张缩略图，InternVL/LLaVA-1.6 的做法）；或**pad 到固定比例**；或**原生分辨率 patch 化**（NaViT）。判断标准：下游是否依赖细节（OCR/小目标 → 必须保分辨率）。

Q30 ★★ 图文 token 比例怎么控制?

A：给一个经验量化：**一张 336×336 图约 576 个 token（patch14）或 256 个（patch16 + 池化）**；LLM 上下文 4096 时，一张图能吃掉 1/7。所以工程上要定"每图 token 预算"（如 256/576/1024），按任务分配——OCR/grounding 给多，闲聊给少。视频更极端：**必须做帧采样 + 时序池化**。

Q31 ★ 模态桥接层选 linear / MLP / Q-Former / Perceiver，怎么选?

A：一句话决策表：

桥接	token 是否压缩	训练成本	什么时候用
linear	不压	最低	极简基线
2 层 MLP	不压	低	**主流选择**（LLaVA 系），token 预算宽松
Q-Former	压到固定数	中（多阶段）	token 预算紧、要多任务对齐（BLIP-2）
Perceiver	压到固定数	中	交错图文、多图（Flamingo）

Q32 ★★ 混合模态训练时，模型"忘记"纯文本能力怎么办?

A：**数据里保留纯文本数据**（通常是图文数据的一定比例，经验值 10%~30%），并且**图文数据与纯文本数据同 batch 混训**；SFT 阶段尤其要注意，否则模型会退化成"只会描述图像"。评测时同时看纯文本 benchmark，别只看多模态分。

S4 幻觉、对齐与评测（12 问）

Q33 ★★★ 多模态大模型的"幻觉"是什么？分几类？

A：模型生成了图像中不存在的信息。两类分法（面试用第二种更专业）： 1. **按对象**：物体幻觉（说有不存在的物体）、属性幻觉（颜色/数量错）、关系幻觉（空间关系错）。 2. **按来源**：**语言先验主导型**（沿用训练语料里的常见搭配，无视图像）、**视觉信息缺失型**（图里看不清/没看到，编一个）、**对齐错误型**（看到了但映射错）。

Q34 ★★★ 幻觉的根因是什么？

A：一句话：**语言先验的强度压过了视觉证据**。具体四层：① 训练数据里图文对噪声大，模型学会"不看图也能猜对"；② 自回归目标鼓励"流畅"而非"忠实"；③ 视觉 token 少（分辨率低/被压缩），证据本身就弱；④ SFT 数据里缺少"我不知道/图里没有"的样本。这条根因和 POPE 里 "adversarial" 设置的设计动机直接相关。

Q35 ★★★ 怎么缓解幻觉？

A：按训练流程分四层： - **数据**：负样本（有图的/没有的）、拒答样本、"不同→对比"样本；提高图文对质量。 - **训练**：加辅助损失（如对比/grounding 监督，强制对齐区域）、视觉 token 加权、加大视觉分辨率。 - **解码**：降低温度、对比解码（VCD，用"加噪图"和"原图"的 logits 差放大视觉贡献）、惩罚高频先验词。 - **后处理/评测**：POPE/HallusionBench 定期体检；RAG 引入外部事实。

你的接口：可以类比讲"我在占用预测里做的 fn/fp 成分解剖（no_hit/wrong_cls/early/late）——把错误拆成分项，才知道该改模型还是改标签"。这套方法论迁移到幻觉上就是"把幻觉按类型统计，而不是只看一个总分"。

Q36 ★★ SFT 数据为什么要"拒绝采样"？

A：让强模型对同一问题采样多条回答，用奖励模型/规则/人工筛掉差的，只保留高质量的那条（best-of-n）。好处：把"生成能力"和"筛选能力"解耦——**用贵的模型筛，用便宜的模型学**。这也是蒸馏型 SFT 数据的常见做法。

Q37 ★★ RLHF 的完整流程？

A：三步：① **SFT** 得到初始策略；② **训练奖励模型 RM**（用人类偏好对 (chosen, rejected) 做 Bradley-Terry 二分类）；③ **用 PPO 优化策略**，奖励 = RM 打分 − β · KL(π ‖ π_ref)（KL 项防止跑偏太远）。多模态里 RM 要吃到图像，所以 RM 本身也是个 VLM。

Q38 ★★★ DPO 和 PPO 的区别？为什么 DPO 更流行？

A：DPO 把"RM + RL"合并成一个**分类式损失**：直接用偏好对 (y_w, y_l) 优化策略，隐含的奖励是 $\beta \cdot \log(\pi(y|x)/\pi_{ref}(y|x))$ 。

- **PPO**：要单独训 RM、要采样、要 4 个模型（policy/value/ref/RM），工程复杂、易不稳。
- **DPO**：不需要 RM、不需要在线采样，一个 loss、两个模型（policy + ref），稳定且便宜。
- 代价：DPO 是**离线**的，没有在线探索，理论上限低于 PPO/在线方法；数据分布外表现差。

追问：GRPO 是什么？→ DeepSeek 系的做法：去掉 value model，对同一个 prompt 采样一组（group）输出，用组内相对优势做基线，省掉 critic 网络，等价于"组内归一化的策略梯度"。

Q39 ★★★ 多模态评测集都测什么？各举一个必背的。

评测集	测什么	一句必背
MMBench	综合能力（20 类能力维度）	用 CircularEval 消除选项顺序偏差
MMMU	大学水平学科推理（图表、公式、考试题）	多学科、多图、需要专业知识
SEED-Bench	生成式理解（图像+视频）	多项选择 + 人工校验
POPE	**物体幻觉**	用"是否存在某物体"的 yes/no 探测幻觉
HallusionBench	幻觉（视觉依赖 + 反事实）	专测"看图说话 vs 靠先验编"

评测集	测什么	一句必背
MM-Vet	综合能力（识别+推理+OCR 混合）	用 GPT-4 打分，能力维度交叉
RefCOCO/RefCOCO+/gRefCOCO	指代定位 grounding	给文本描述输出框，准确率/IoU
VideoMME / LongVideoBench	视频理解（短/中/长，有无字幕）	长视频显式考察时序与检索
TextVQA / DocVQA / ChartQA	OCR 与文档/图表理解	工业落地最常用

Q40 ★★ POPE 的三个设置是什么？为什么这么设计？

A：**random**（随机选不存在物体）、**popular**（选数据集中高频但图里没有的物体）、**adversarial**（选和图中物体共现频率高、最容易骗过模型的物体）。指标是 Accuracy / Precision / Recall / **F1** 和 yes 比例。设计动机：随机负样本太简单，会上报虚高分数，**adversarial 才是真考验"是否真在看图"**。

Q41 ★★ MMBench 的 CircularEval 是什么？为什么需要？

A：把一道多选题的选项**循环移位多次**，每次让模型回答，只有全部正确才算对。因为 VLM 对选项位置有强烈偏好（比如总选 A/B），不移位会高估能力。这是"评测口径"的经典例子——**面试时讲这个，比背指标更能证明你读过评测细节**。

Q42 ★★ 评测里常见的坑有哪些？

A：五个：① **数据泄漏**（训练集和评测集图像近重复，尤其 CLIP 系预训练数据几乎覆盖全网）；② **prompt 敏感**（换个模板差 5 个点）；③ **选项偏差**（要 CircularEval / 打乱）；④ **指标口径不一致**（BLEU/CIDEr 对生成式评测不可靠，所以现代评测多用 GPT 打分或人工，但 GPT 打分本身有偏差）；⑤ **只报总分不报分项**（真实能力要看分项，比如 OCR 强但推理弱）。

Q43 ★★ grounding 任务怎么评测？

A：输入"图中的白色小狗"，模型输出框或 mask，和 GT 比 **IoU / 点命中**。指标：RefCOCO 用 Acc@0.5（IoU>0.5 的比例），也常用**点级准确率**（输出点落在 GT mask 内的比例，更宽容）。难点：**指代歧义**（多只同类物体）、**否定/关系描述**（"不在桌上的那个"）。

Q44 ★ 为什么要关心"纯文本能力不退化"？

A：多模态模型最终是个 LLM，用户会问纯文本问题。若 SFT 只喂图文数据，模型的语言能力、指令遵循、长文组织都会退化（catastrophic forgetting）。做法：混入纯文本指令数据、评测时同时跑文本 benchmark、必要时用 LoRA 限制参数漂移。

S5 视频理解与生成（10 问）

Q45 ★★★ 视频的 token 数量为什么是灾难？怎么压？

A：算一下就明白：一帧 224×224/patch16 = 196 token，**32 帧就是 6,272 token**，还没算文本；1 分钟 30fps 有 1800 帧 → 35 万 token。**必须压**。

四条路：① **帧采样**（均匀/关键帧/内容自适应，常见 8~64 帧）；② **空间压缩**（每帧先池化/pixel shuffle）；③ **时间压缩**（相邻帧特征做时序池化、3D VAE、token merge）；④ **分层+检索**（长视频先摘要再检索片段，LongVA/Video-LLaVA 那类）。工程口诀：**"先降帧率，再降分辨率，最后压在时序维上"**。

Q46 ★★ 视频里的时序信息怎么建模？

A：三种主流：① **逐帧独立编码 + 序列拼接**（最简单，靠 LLM 自己学时序，需要时序位置编码区分帧）；② **时空联合 patch**（3D ViT / VideoMAE，把 t 维也切 patch）；③ **时空聚合模块**（在视觉塔里做帧间 attention 或 pooling，输出固定帧数）。位置编码上常用 **3D RoPE（时间+高+宽）**，Qwen2-VL 就是 M-RoPE。

Q47 ★★ 长视频（1 小时以上）怎么做？

A：核心思想是**不要把所有帧塞进上下文**，而是"摘要 + 检索 + 分层"： 1. 关键帧/场景切分（用变化检测或聚类）； 2. 每段单独编码成紧凑描述（caption/embedding）； 3. 提问时先检索相关段（embedding 检索或轻量模型打分），只把相关段的细节 token 送进 LLM； 4. 需要全局理解时用分层摘要（段→章）。

你的接口：你的 scene-aligned 时序切分（避免跨场景污染）+ 时序历史缓存控制，和这个"分段+检索"思路同构，可以顺势讲。

Q48 ★★ 视频理解评测看什么?

A：**VideoMME**（短/中/长视频 × 有无字幕，显式暴露长视频退化）、**LongVideoBench**（长上下文）、**MVBench**（时序动作）、**EgoSchema**（第一视角长视频）。关键不是分数而是**分档看**：短视频好、长视频崩，是普遍的现状——所以要报"分时长档"的结果。

Q49 ★★ 扩散模型的前向和反向过程各是什么?

A：- **前向**（加噪，无参数）：$x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1-\bar{\alpha}_t)} \cdot \varepsilon$，T 步（典型 1000 步）把数据变成纯噪声，马尔可夫链。 - **反向**（去噪，学网络）：训练一个网络 $\varepsilon_\theta(x_t, t)$ 预测噪声，损失是 $||\varepsilon - \varepsilon_\theta(x_t, t)||^2$（也可以是预测 x_0 或 v）。采样时从纯噪声逐步去噪。

一句话本质：**扩散模型 = 学一个"从噪声回到数据"的分数函数（score）**。

Q50 ★★ DDPM 和 DDIM 的区别?

A：DDPM 的反向是**随机**马尔可夫过程（每步加噪），要 1000 步；DDIM 构造一个**确定性**（或可控随机）的非马尔可夫采样过程，**可以用 20~50 步得到近似结果**，且同一噪声种子结果可复现。后来的 DPM-Solver、uniPC 把它做成 ODE 求解器，10 步左右就够。**工程意义：采样步数是生图/生视频的延迟主因**。

Q51 ★★★ CFG（Classifier-Free Guidance）是什么?

A：训练时把条件（文本）以一定概率置空，让同一个网络同时学"有条件"和"无条件"；采样时把两者外推：

$$\varepsilon = \varepsilon_{uncond} + s \cdot (\varepsilon_{cond} - \varepsilon_{uncond})$$

s 就是 guidance scale（常用 5~9）。**s 大 → 更贴合文本但多样性下降、易过饱和；s=1 → 等于纯条件采样。**

追问：为什么叫 classifier-free? → 早期用单独训的分类器梯度做引导（Classifier Guidance），后来改成同一个网络的条件/无条件差，不需要分类器。

Q52 ★★ Latent Diffusion（Stable Diffusion）的关键设计?

A：不在像素空间扩散，而是先用 VAE 把图压到**低维 latent**（如 512×512×3 → 64×64×4，压缩 48 倍），在 latent 上做扩散，最后 VAE 解码回像素。好处：算力下降一个量级以上，让消费级卡能训。代价：VAE 是瓶颈（细节/文字渲染差，所以后来的 SD3/Flux 改 VAE 或换架构）。条件注入用 **cross-attention**（文本 token 作为 K/V）——**这里和 LLM 的多模态融合是同一个机制，可以顺势讲**。

Q53 ★ DiT 是什么? 统一理解与生成为什么难?

A：DiT 把扩散的 U-Net 换成 **Transformer**（patch 化 + adaLN 注入时间/条件），scaling 行为更好，是 SD3/Flux/Sora 系的基础。统一模型的难点：**理解要判别式语义特征，生成要像素级重建细节**，两者目标冲突；路线有"共享 Transformer + 不同 head"（Transfusion、Emu3 用离散 token 统一）、"生成用扩散头、理解用 LM 头"。目前没有公认最优解——**答"这是开放问题"比硬编答案好**。

Q54 ★★ 视频生成比图像生成难在哪?

A：四条：① **时序一致性**（闪烁、形变、物体消失）；② **数据**（高质量视频+文本对远少于图像）；③ **算力/显存**（3D VAE + 时空注意力，长度线性涨）；④ **运动可控性**（镜头运动 vs 物体运动要分开控制，常用轨迹/光流/相机参数条件）。工程上还有"首尾帧控制、参考图一致性"这类产品化需求。

S6 推理部署与工程（10 问）

Q55 ★★★ 推理时 KV cache 怎么算? 为什么它是瓶颈?

A：每层每个 token 要缓存 K 和 V：

```
显存 ≈ 2 × n_layers × n_kv_heads × d_head × seq_len × batch × 精度字节
```

例：32 层、32 头、d_head=128、fp16、seq 2048、batch 1 → 2×32×32×128×2048×2 ≈ **1.07 GB**，batch 和长度都线性叠上去。

它是瓶颈的原因：**decode 阶段每一步都要读一遍整个 cache**，是**访存密集**而不是计算密集，所以长上下文时 GPU 利用率很低。解法：GQA/MQA（`n_kv_heads` 变小）、KV 量化（INT8/INT4）、PagedAttention（消除碎片）、滑动窗口/丢弃远距离、prefix cache 复用。

Q56 ★★★ Prefill 和 Decode 两个阶段的差异?

A:

	Prefill（读 prompt）	**Decode（逐 token 生成）**
并行度	整段并行，**计算密集**（大 GEMM）	每步 1 个 token，**访存密集**
瓶颈	算力 / FLOPS	HBM 带宽（要读全部权重+KV cache）
优化	FlashAttention、大 batch、融合算子	KV cache 压缩、投机解码、连续批处理、量化

面试价值："端侧延迟主要花在 prefill（图 token 多）还是 decode（输出长）要分开看"——这句话能立刻显出你有实测概念。**你的真实数据**：MDC610 上做算子解耦+向量化把误检从 约 5% 降到约 1.4%，可以顺势讲"我做过 prefill 侧（视觉塔+投影）的算子级优化"。

Q57 ★★ PagedAttention / vLLM 解决什么问题?

A：传统实现给每个请求预分配连续 KV 显存，长度未知导致**大量碎片和浪费（实测浪费可到 60%+）**。PagedAttention 把 KV cache 切成固定大小的 block，用页表映射，**按需分配、可共享（同一 prompt 的多个采样共享前缀）**，配合 **continuous batching**（请求完成就换新请求进 batch，不等最慢的）把吞吐提升数倍。

Q58 ★★ 投机解码（speculative decoding）是什么?

A：用一个小模型（或 n-gram/检索）先草拟 k 个 token，大模型**并行验证**这 k 个，接受前缀匹配的部分、拒绝第一个不匹配处。因为验证是并行的（一次 forward 出 k 个位置的 logits），**在不改变输出分布的前提下加速**（严格等价，这是它的理论保证）。典型加速 1.5~3×，取决于草稿命中率。

Q59 ★★ 量化怎么选? INT8 / INT4 各有什么代价?

A：权重 INT8 基本无损（W8A8/W8A16 常用）；**INT4 权重会出现明显精度损失**，需要算法补偿：**GPTQ**（逐层量化 + Hessian 补偿）、**AWQ**（保护重要通道，激活感知）、**SmoothQuant**（把激活的难度迁到权重上）。还有 KV cache 量化（长上下文更划算）。判断标准：先看任务对长尾/数值敏感的程度（OCR、数学推理对量化更敏感）。

Q60 ★★ 蒸馏在多模态里怎么做?

A：三条路：① **logit 蒸馏**（学生对齐教师的输出分布，加温度 T）；② **特征蒸馏**（对齐中间层/token 表示，多模态常用"对齐教师看同一张图后的视觉 embedding"）；③ **数据蒸馏**（用教师生成的问答对训学生，最常用、最省事）。端侧落地常见组合：**小视觉塔 + 小 LLM + 教师蒸馏数据 + 量化**。

Q61 ★★ 端侧多模态模型最难的三件事?

A： 1. **分辨率 vs 算力**：端侧要看清（OCR/小目标）就得高分辨率，而分辨率直接变成 token 和算力——这是根本矛盾。 2. **显存/内存**：手机/车机的可用内存是 GB 级，KV cache + 权重 + 激活都要算进去。 3. **算子与后端**：不是所有算子都能落到 NPU/GPU（attention 的动态形状、动态分辨率、nms/topk），要**改写算子或换等价语义**。

你的接口（这是你最强的差异化）："我在 MDC610 上做过完整的 ONNX→ATC/AOE→`.om` 链路，对不支持算子做了解耦 + OpenCV 向量化重写，端侧误检 约 5%→约 1.4%。所以端侧多模态我也会先问三个问题：**目标 SoC 的算子白名单、内存预算、以及是否允许动态分辨率**。"

Q62 ★★★ 昇腾（Ascend）生态你了解多少？——送分题，必须答满

A：分层说： - **硬件**：Ascend 310/910B（推理/训练），达芬奇架构（Cube 矩阵单元 + Vector 单元）。 - **软件栈**：**CANN**（异构计算架构，对标 CUDA）→ **ATC**（模型转换：ONNX/PB → `.om` 离线模型）→ **AOE**（自动调优，选 tiling/算子实现）→ **AscendCL**（推理 API）。 - **框架**：**MindSpore**（自研）/ PyTorch 通过 torch_npu 适配。 - **工程坑**：算子不支持要拆或替换；动态 shape 支持有限，常需分档固定 shape；精度对齐要做（fp16 溢出、算子融合顺序差异）；内存要手工规划（模型/工作空间）。

话术："我做过完整的 ONNX(opset11/16)→ATC/AOE→MDC610 `.om` 落地，对 AscendCL 的输入输出 binding、内存对齐、算子不支持时的等价改写都有实操。"→ **这是华为面试里最能加分的一段，因为大部分算法候选人只会写 train.py。**

Q63 ★ 端侧时延怎么拆？怎么定位瓶颈？

A：拆成四段并分别打点：① **预处理**（resize/归一化/拼接，CPU 上很贵，别忽略）→ ② **视觉塔**（ViT 前向，可用 batch/半精度）→ ③ **projector + prefill**（LLM 一次前向处理所有图文 token）→ ④ **decode**（每 token 一次，按输出长度线性）。

定位方法：先测各段独立耗时 + 用 nsys/推理框架的 profile 看算子占比；常见结论是 **prefill 被图像 token 拖累、decode 被 KV cache 带宽拖累、预处理被 Python/CPU 拷贝拖累（用 pinnedMemory/零拷贝可省几十毫秒）**。

Q64 ★ 多模态推理服务的吞吐怎么提？

A：五个手段：① **统一 batch 内的 shape**（动态分辨率会让 batch 里 shape 不一，要么 padding 要么分桶）；② **连续批处理**（不等最长序列）；③ **prefix/图像特征缓存**（同一张图多轮问答，视觉塔的结果缓存重用，这是多模态特有的收益）；④ **量化 + 投机解码**；⑤ **视觉塔与 LLM 异构并行**（视觉塔放另一张卡/另一个流，与 decode 重叠）。

S7 把你的项目接进多模态叙事 + 临场策略（10 问）

Q65 ★★★ 「你做的是感知，凭什么做多模态大模型？」怎么答？

A：三段式，**不要说"我可以学"**： 1. **做过的同构问题**："我做的视觉+激光融合本质是跨模态对齐：两种异构传感器（图像是稠密透视、激光是稀疏度量）在统一 BEV 网格上做门控融合，处理的核心难点和 VLM 里的图文对齐同源——**都是把不同分布、不同粒度的表征映射到一个可比较的空间**。" 2. **可迁移的工程能力**："大规模训练调优（8 卡训练集群、显存墙下的结构重构、loss 崩溃的三连排查）、数据生产线与评测口径（25 万场景/百万关键帧、双 seed 噪声地板）、端侧部署（ONNX→ATC→MDC610）。这三件事恰好是 VLM 落地最缺的能力——**模型结构论文里都有，但数据、训练稳定性、部署是真功夫**。" 3. **明确承认缺口 + 已做的补课**："VLM 的 SFT/偏好对齐我没在生产里做过，所以我按 LLaVA 两阶段在自己的数据上复现过、也读了 Qwen-VL/InternVL 的报告，重点看它们的 token 压缩和数据配比。"

Q66 ★★★ 你的视觉-激光融合 vs 多模态的 cross-attention，有什么关系？

A："我的融合是**结构化的跨模态交互**：两路特征在 BEV 网格上对齐后，用 SE 通道门控做重加权，再由卷积融合。VLM 里的 cross-attention 是**用注意力动态决定'从另一模态取什么'**，表达力更强但计算更贵（$O(N^2)$）。

如果让我改造，我会把静态门控换成 **query-based cross-attention**：BEV query 作 Q、图像特征作 K/V，这样融合权重是按位置和内容动态生成的——**这正好是 BEVFormer 和 Flamingo/Q-Former 的共同思路**。"

（**这段是全场最高分的回答，把两个领域用同一个机制连起来了**）

Q67 ★★ 你的 BEV query 和 Q-Former 的 query 是一回事吗？

A："形式上同类：都是**固定数量、可学习、通过 cross-attention 从高冗余输入里抽信息**。差别在约束：BEV query 有**强几何先验**（每个 query 对应一个固定的世界坐标格子，靠相机内外参做投影），所以 token 的位置是"物理正确"的；Q-Former 的 query 没有几何约束，纯靠数据学。

前者适合需要**度量精度**的任务（占用、检测），后者适合**语义检索**（"图里有什么"）。如果要给 VLM 加空间精度，用带几何先验的 query（3D grounding、SpatialVLM 一类工作）是自然方向。"

Q68 ★★ 你的时序建模经验怎么迁移到视频多模态?

A："我做过两件相关的事：按 scene 对齐切时序段（避免跨场景污染）、维护时序历史缓存（在 mIoU 与显存之间做权衡）。换成视频理解，对应的问题是**帧采样策略**和**历史 token 预算**：帧采样的关键是'别把同一场景的冗余帧重复喂进去'，历史 token 的关键是'压缩方式要与任务相关'——我倾向于先做场景切分 + 关键帧，再做时序池化，这和我在 OCC 上的做法同构。"

Q69 ★★ 数据生产线经验怎么讲成 VLM 的加分项?

A："多模态模型的上限基本由数据决定，而我在'数据-评测-归因'这条链上有完整实操：① **大规模生产**（数万个场景 / 百万级关键帧，含三态标注）；② **口径治理**（发现过转换器两个方向相反的 filter 版本导致 mask 体积只有 约 0.64 倍、mIoU 降约 3 个点 的隐性 bug，用逐元素对比 + IoU=1.0 锁定）；③ **评测设计**（万级帧协议 + 双 seed 噪声地板，把'模型变好'和'评测抖动'分开）。

迁到 VLM：数据配比、去重、caption 重写、幻觉负样本构造、以及'分项评测而不是只看总分'，方法论完全一致。"

Q70 ★★ 端侧部署经验怎么讲?

A：见 Q61/Q62。核心三句：**ONNX→ATC/AOE→ `.om` 全链路实操过；不支持算子做等价改写（Decode/TopK/NMS 解耦 + OpenCV 向量化）；有实测指标（ROI 超分误检 约 5%→约 1.4%）。**

Q71 ★ 具身智能 / VLA / 空间智能你了解吗?

A：一句话版：**VLA = 视觉语言模型 + 动作输出头**（RT-2、OpenVLA、π0 这类），把"看图 + 听指令 → 输出动作 token/轨迹"。关键难点：动作数据的稀缺与异构（不同机器人本体）、实时性（要 10~100Hz 控制频率）、空间精度（需要 3D 表征而不是 2D token）。

你的优势：**你是所有候选人里少数真做过 3D 几何、占有栅格、标定的人**——"具身落地最缺的就是 3D 空间理解与传感器标定，这两块是我的强项"。这句话会让面试官记住你。

Q72 ★★★ 遇到完全不会的问题怎么答？（临场策略，价值极高）

A：三步，别硬编： 1. **确认范围**："您问的是训练侧还是推理侧？"（把开放问题变窄，争取思考时间）； 2. **给出你确定的相邻知识**："这块我没在生产里做过，但我知道它的相邻问题是这样解决的……"（展示知识网而不是空白）； 3. **给出验证路径**："如果让我做，我会先在小规模上做一个对照实验，用 XX 指标卡住，再放大。"（展示方法论）。

绝对不要：瞎编数字、把"听说过"说成"做过"、被追问就沉默。**面试官容忍"不会"，不容忍"编"。**

Q73 ★★ 反问环节问什么?

A：三个显示你在想落地的问题： 1. 团队的多模态模型主要做哪个模态组合（图文/视频/点云-图），当前瓶颈在数据、结构还是推理成本？ 2. 端侧和云侧的比例大概多少？时延/算力预算是多少，昇腾侧是自研 ATC 链路还是 MindSpore 栈？ 3. 新人前三个月最可能被派到哪一块（数据管线 / 训练 / 评测 / 部署）？

Q74 ★★★ 红线清单（宁可少说，不可说错）

1. **不说自己训过 LLM/多模态大模型**。说"做过同构的跨模态对齐与大规模训练调优，VLM 侧在系统补课"。
2. **不报参数量**（百万级参数量 这种会被"这也叫大模型"直接打穿）。改说结构设计意图与单变量控制。
3. **不说论文已发表**。
4. **不说做过 SLAM**（只说视觉几何、位姿对齐、ICP、4-DOF 同位变换）。
5. **不跨协议比指标**。被问延迟/精度，先报口径（帧数、平台、是否含预处理）再报数。
6. **不把"读过论文"说成"复现过"**。可以说"读过并做过最小复现"。
7. **不确定的数字给量级**（"约 4 亿图文对"而不是"400,132,000 对"）。

附：按 30 秒精度划分的优先级（复习用）

优先级	题目	说明
必须脱口而出（15 题）	Q1 Q6 Q9 Q14 Q20 Q21 Q22 Q25 Q26 Q34 Q39 Q40 Q55 Q56 Q62	这些答不上来基本判定"没做过多模态"
要能讲 1 分钟（25 题）	Q3 Q4 Q5 Q10 Q11 Q12 Q13 Q15 Q23 Q24 Q27 Q30 Q33 Q35 Q37 Q38 Q41 Q42 Q45 Q48 Q49 Q51 Q57 Q59 Q63	讲到"原因 + 一个例子"即可
能接住就行（其余）	剩下全部	听懂了、能接一句，不要把不确定的说满

复习节奏建议：面试前 3 天每天过一遍"必须脱口而出 15 题"；面试前 1 小时只看这一页的表格。